You said:

```
import random
import pygame
from game.text_box import TextBox

class GameEngine:
    def __init__(self, width, height):
        self.width = width
        self.height = height

        self.words = ["PYTHON", "PYGAME", "PLANET", "ROCKET", "GALAXY", "STREAM", "PUZZLE",
"ALGORITHM"]
        self.secret_word = ""
        self.scrambled_word = ""

        self.score = 0
        self.feedback_msg = "Unscramble the letters above!"
        self.feedback_color = (210, 215, 225)

        self.input_box = TextBox(width // 2 - 130, 210, 160, 46)
        self.submit_btn = pygame.Rect(width // 2 + 45, 210, 95, 46)

        self.font_title = pygame.font.SysFont(None, 40)
        self.font_word = pygame.font.SysFont(None, 52)
        self.font_msg = pygame.font.SysFont(None, 26)
        self.font_btn = pygame.font.SysFont(None, 24)

        self.next_round()

    def scramble_string(self, word):
```

```
        letters = list(word)
        while True:
            random.shuffle(letters)
            shuffled = "".join(letters)
            if shuffled != word or len(word) <= 1:
                return shuffled

    def next_round(self):
        self.secret_word = random.choice(self.words)
        self.scrambled_word = self.scramble_string(self.secret_word)
        self.input_box.clear()

    def submit_guess(self):
        guess = self.input_box.text.strip().upper()
        if not guess:
            self.feedback_msg = "Type a word before submitting!"
            self.feedback_color = (240, 170, 50)
            return

        is_correct = (guess == self.scrambled_word)

        if is_correct:
            self.score += 1
            self.feedback_msg = f"CORRECT! '{self.secret_word}' is right."
            self.feedback_color = (80, 230, 110)
            self.next_round()
        else:
            self.feedback_msg = "WRONG GUESS! Try again."
            self.feedback_color = (240, 80, 80)
            self.input_box.clear()
```

```
def handle_event(self, event):
    self.input_box.handle_event(event)

    if event.type == pygame.KEYDOWN and event.key == pygame.K_RETURN:
        self.submit_guess()
    elif event.type == pygame.MOUSEBUTTONDOWN and event.button == 1:
        if self.submit_btn.collidepoint(event.pos):
            self.submit_guess()

def update(self):
    pass

def render(self, screen):
    screen.fill((26, 30, 38))

    title_surf = self.font_title.render("Word Scramble Arena", True, (245, 245, 245))
    screen.blit(title_surf, (self.width // 2 - title_surf.get_width() // 2, 25))

    score_surf = self.font_msg.render(f"Score: {self.score}", True, (255, 220, 80))
    screen.blit(score_surf, (self.width // 2 - score_surf.get_width() // 2, 70))

    spaced_letters = "  ".join(self.scrambled_word)
    scramble_surf = self.font_word.render(spaced_letters, True, (100, 200, 255))
    screen.blit(scramble_surf, (self.width // 2 - scramble_surf.get_width() // 2, 130))

    self.input_box.render(screen)

    pygame.draw.rect(screen, (50, 150, 85), self.submit_btn, border_radius=6)
    pygame.draw.rect(screen, (220, 220, 220), self.submit_btn, width=2, border_radius=6)
    btn_text = self.font_btn.render("SUBMIT", True, (255, 255, 255))
    screen.blit(btn_text, (self.submit_btn.centerx - btn_text.get_width() // 2,
```

self.submit_btn.centery - btn_text.get_height() // 2))

        feedback_surf = self.font_msg.render(self.feedback_msg, True, self.feedback_color)
        screen.blit(feedback_surf, (self.width // 2 - feedback_surf.get_width() // 2, 285))

shwetapatil7
62_scramble
Repository navigation
Code
Pull requests
Agents
Actions
Projects
Wiki
Security and quality
Insights
Settings
shwetapatil7
62_scramble
Public
forked from SETAPESU26/62_scramble
Go to file
t
T
This branch is up to date with SETAPESU26/62_scramble:main.
kritikarungta633-code
kritikarungta633-code
Update game_engine.py
7ae8f8f
.

2 weeks ago
Name
game
Update game_engine.py
2 weeks ago
README.md
Update README.md
2 weeks ago
main.py
Add files via upload
3 weeks ago
Repository files navigation
README
Word Scramble Repair Lab
This project is an interactive anagram deduction word puzzle game using Pygame. It introduces students to string permutation, randomized list shuffling, uppercase letter sanitization, and text-box widget integration within an object-oriented codebase.
What's Provided
A working Word Scramble game with:

A built-in dictionary pool of words chosen at random each round
A scrambling algorithm that randomizes letter order while ensuring the scrambled version differs from the original word
A custom TextBox input component handling alphabetic keystrokes, automatic uppercase conversion, and backspace
Interactive guess submission via the Return / Enter key or clicking the SUBMIT button
Real-time score tracking, spaced letter displays, and color-coded status messaging
It has one deliberate bug and three optional features left as tasks to implement. You are expected to analyze, interact with an AI assistant, and complete/fix the game to make it fully functional and more interesting.

Use an LLM (e.g. ChatGPT or Claude) as your debugging and pair-programming partner for this lab.

Getting Started

Setup

Make sure you have Python 3.10+ installed.

Install dependencies:

```
pip install pygame
```

Run the game:

```
python main.py
```

Controls: Type letters into the text box and press Return (or click SUBMIT) to submit your unscrambled word

Tasks to Complete

Each task must be completed using an iterative process involving LLM suggestions and your critical code review.

Task 1: Fix the guess comparison validation bug

Typing the correct unscrambled word is rejected as incorrect, while re-entering the scrambled letter string registers as a valid answer. Correct the guess verification logic so submitted words are validated against the actual secret target word rather than the scrambled display text.

Task 2: Implement a progressive hint system

Challenging words can leave players stuck with no path forward. Add an interactive hint feature that reveals individual letters of the secret word in their correct positions upon request, applying a proportional score penalty for each hint used.

Task 3: Implement a active round countdown timer

Players currently have unlimited time to ponder anagrams. Add a visible countdown timer bar to the round that ticks down during active play, automatically revealing the word and advancing to the next round if time runs out before a correct answer is submitted.

Task 4: Implement Interactive Letter Tile Sorting
Scrambled letters are currently presented as a static text string. Replace the static label with interactive, graphical letter tiles that players can click or rearrange to physically experiment with different letter sequences before submitting a guess.

Expected Behavior
At the start of each round, a word is chosen and displayed with its letters jumbled.
Typing the correctly unscrambled word into the input box awards 1 point, displays a success message, and advances to a new word.
Typing an incorrect word displays a warning message and clears the input box for retry without advancing the round.
Empty submissions trigger a prompt without penalty.
Folder Structure
word_scramble/
├── game/
│ ├── game_engine.py
│ └── text_box.py
├── main.py
└── README.md
Submission Checklist
Submission is only the following three things:

[] A 10-second video of gameplay before your changes, showing the bug/broken behavior
[] A 10-second video of gameplay after your changes, showing the bug fixed and the new features working
[] The Chat/LLM used page link, with the complete chat history
About

No description, website, or topics provided.
Resources
Readme

Activity
Stars
0 stars
Watchers
0 watching
Forks
0 forks
Releases
No releases published
Create a new release
Packages
No packages published
Publish your first package
Contributors
No contributors
Languages
Python
100%
Suggested workflows
Based on your tech stack

Publish Python Package logo
Publish Python Package
Publish a Python Package to PyPI on release.
By GitHub Actions
Python package logo
Python package
Create and test a Python package on multiple Python versions.
By GitHub Actions
Pylint logo
Pylint

Lint a Python application with pylint.
By GitHub Actions
More workflows

use this and complte thatr code

ChatGPT said:

Shweta, based on your GitHub repository `62_scramble` and its README, you need to fix the existing bug and implement **three new features** in your Word Scramble game.

Your current code has one important error:

```
is_correct = (guess == self.scrambled_word)
```

It compares the player's answer with the scrambled letters instead of the original word. It should be:

```
is_correct = (guess == self.secret_word)
```

Below is a complete replacement for `game/game_engine.py` that includes all four tasks while keeping your existing `TextBox` class and `main.py` structure.

Replace `game/game_engine.py` with this complete code

```
import random
import pygame
```

```
from game.text_box import TextBox


class GameEngine:
    def __init__(self, width, height):
        self.width = width
        self.height = height

        self.words = [
            "PYTHON", "PYGAME", "PLANET", "ROCKET",
            "GALAXY", "STREAM", "PUZZLE", "ALGORITHM"
        ]

        self.secret_word = ""
        self.scrambled_word = ""
        self.tile_letters = []
        self.selected_tile = None

        self.score = 0.0
        self.hints_used = 0
        self.revealed_positions = set()

        self.round_duration = 20  # seconds
        self.round_start_time = pygame.time.get_ticks()
        self.round_over_time = None
        self.game_over_pending = False

        self.feedback_msg = "Unscramble the letters above!"
        self.feedback_color = (210, 215, 225)

        center = width // 2
```

```
    # Input and buttons
    self.input_box = TextBox(center - 220, 255, 200, 46)
    self.submit_btn = pygame.Rect(center - 5, 255, 100, 46)
    self.hint_btn = pygame.Rect(center + 110, 255, 90, 46)

    # Fonts
    self.font_title = pygame.font.SysFont(None, 40)
    self.font_word = pygame.font.SysFont(None, 34)
    self.font_msg = pygame.font.SysFont(None, 24)
    self.font_btn = pygame.font.SysFont(None, 22)
    self.font_tile = pygame.font.SysFont(None, 32)
    self.font_small = pygame.font.SysFont(None, 20)

    self.next_round()

def scramble_string(self, word):
    letters = list(word)

    if len(letters) > 1:
        while True:
            random.shuffle(letters)
            shuffled = "".join(letters)

            if shuffled != word:
                return shuffled

    return word

def next_round(self):
    self.secret_word = random.choice(self.words)
```

```
    self.scrambled_word = self.scramble_string(self.secret_word)

    self.tile_letters = list(self.scrambled_word)
    self.selected_tile = None

    self.hints_used = 0
    self.revealed_positions = set()

    self.input_box.clear()

    self.round_start_time = pygame.time.get_ticks()
    self.round_over_time = None
    self.game_over_pending = False

def get_time_left(self):
    elapsed = (
        pygame.time.get_ticks() - self.round_start_time
    ) / 1000

    return max(0, self.round_duration - elapsed)

def get_current_guess(self):
    # Use typed input when provided.
    typed_guess = self.input_box.text.strip().upper()

    if typed_guess:
        return typed_guess

    # Otherwise, use the order of the letter tiles.
    return "".join(self.tile_letters)
```

```
def submit_guess(self):
    if self.game_over_pending:
        return

    guess = self.get_current_guess()

    if not guess:
        self.feedback_msg = "Type or arrange a word first!"
        self.feedback_color = (240, 170, 50)
        return

    # TASK 1: Compare with the original secret word.
    if guess == self.secret_word:
        # Each hint costs 0.25 points.
        points_earned = max(0, 1.0 - 0.25 * self.hints_used)
        self.score += points_earned

        self.feedback_msg = (
            f"CORRECT! {self.secret_word} | "
            f"+{points_earned:.2f} points"
        )
        self.feedback_color = (80, 230, 110)

        # Start a fresh round.
        self.next_round()

    else:
        self.feedback_msg = "WRONG GUESS! Try again."
        self.feedback_color = (240, 80, 80)
        self.input_box.clear()
```

```
def give_hint(self):
    if self.game_over_pending:
        return

    # Reveal one additional letter in its correct position.
    hidden_positions = [
        i for i in range(len(self.secret_word))
        if i not in self.revealed_positions
    ]

    if not hidden_positions:
        self.feedback_msg = "All letters have been revealed!"
        self.feedback_color = (240, 170, 50)
        return

    position = random.choice(hidden_positions)
    self.revealed_positions.add(position)

    self.hints_used += 1
    self.score = max(0, self.score - 0.25)

    self.feedback_msg = (
        f"Hint revealed! -0.25 points "
        f"({len(self.revealed_positions)}/{len(self.secret_word)})"
    )
    self.feedback_color = (255, 210, 90)

def handle_tile_click(self, position):
    if self.game_over_pending:
        return
```

```
tile_width = 44
tile_gap = 8
total_width = (
    len(self.tile_letters) * tile_width
    + (len(self.tile_letters) - 1) * tile_gap
)
start_x = (self.width - total_width) // 2
tile_y = 155

for i in range(len(self.tile_letters)):
    rect = pygame.Rect(
        start_x + i * (tile_width + tile_gap),
        tile_y,
        tile_width,
        48
    )

    if rect.collidepoint(position):
        if self.selected_tile is None:
            self.selected_tile = i
        elif self.selected_tile == i:
            self.selected_tile = None
        else:
            # TASK 4: Swap the two selected letter tiles.
            self.tile_letters[self.selected_tile], \
                self.tile_letters[i] = (
                    self.tile_letters[i],
                    self.tile_letters[self.selected_tile]
                )

            self.selected_tile = None
```

```
            return

    def handle_event(self, event):
        if not self.game_over_pending:
            self.input_box.handle_event(event)

        if event.type == pygame.KEYDOWN:
            if event.key == pygame.K_RETURN:
                self.submit_guess()

        elif event.type == pygame.MOUSEBUTTONDOWN and event.button == 1:
            if self.submit_btn.collidepoint(event.pos):
                self.submit_guess()

            elif self.hint_btn.collidepoint(event.pos):
                self.give_hint()

            else:
                self.handle_tile_click(event.pos)

    def update(self):
        # TASK 3: End the round when its countdown reaches zero.
        if self.game_over_pending:
            elapsed = pygame.time.get_ticks() - self.round_over_time

            if elapsed >= 1800:
                self.next_round()

            return
```

```
    if self.get_time_left() <= 0:
        self.game_over_pending = True
        self.round_over_time = pygame.time.get_ticks()

        self.feedback_msg = (
            f"TIME UP! The correct word was {self.secret_word}"
        )
        self.feedback_color = (255, 100, 100)

        self.input_box.clear()

def render(self, screen):
    screen.fill((26, 30, 38))

    # Title
    title = self.font_title.render(
        "Word Scramble Arena", True, (245, 245, 245)
    )
    screen.blit(
        title,
        (self.width // 2 - title.get_width() // 2, 20)
    )

    # Score
    score_text = self.font_msg.render(
        f"Score: {self.score:.2f}", True, (255, 220, 80)
    )
    screen.blit(
        score_text,
        (self.width // 2 - score_text.get_width() // 2, 65)
    )
```

```
# TASK 3: Countdown timer and timer bar
time_left = self.get_time_left()

timer_text = self.font_small.render(
    f"Time left: {time_left:.1f}s",
    True,
    (245, 245, 245)
)
screen.blit(
    timer_text,
    (self.width // 2 - timer_text.get_width() // 2, 91)
)

bar_width = min(420, self.width - 40)
bar_x = (self.width - bar_width) // 2
bar_y = 116

pygame.draw.rect(
    screen, (65, 70, 82),
    (bar_x, bar_y, bar_width, 10),
    border_radius=5
)

remaining_width = int(
    bar_width * time_left / self.round_duration
)

if remaining_width > 0:
    timer_color = (
        (80, 220, 110) if time_left > 7
```

```
        else (255, 170, 50) if time_left > 3
        else (255, 70, 70)
    )

    pygame.draw.rect(
        screen, timer_color,
        (bar_x, bar_y, remaining_width, 10),
        border_radius=5
    )

# TASK 4: Interactive letter tiles
tile_width = 44
tile_gap = 8
total_width = (
    len(self.tile_letters) * tile_width
    + (len(self.tile_letters) - 1) * tile_gap
)
start_x = (self.width - total_width) // 2

for i, letter in enumerate(self.tile_letters):
    rect = pygame.Rect(
        start_x + i * (tile_width + tile_gap),
        155,
        tile_width,
        48
    )

    tile_color = (
        (220, 150, 45)
        if i == self.selected_tile
        else (55, 100, 145)
```

```
        )

        pygame.draw.rect(
            screen, tile_color, rect, border_radius=6
        )
        pygame.draw.rect(
            screen, (190, 210, 230),
            rect, width=2, border_radius=6
        )

        letter_surface = self.font_tile.render(
            letter, True, (255, 255, 255)
        )

        screen.blit(
            letter_surface,
            (
                rect.centerx - letter_surface.get_width() // 2,
                rect.centery - letter_surface.get_height() // 2
            )
        )

    instruction = self.font_small.render(
        "Click two tiles to swap them, or type your answer below.",
        True,
        (190, 200, 215)
    )
    screen.blit(
        instruction,
        (self.width // 2 - instruction.get_width() // 2, 210)
    )
```

```
# TASK 2: Progressive hint display
hint_pattern = " ".join(
    letter if i in self.revealed_positions else "_"
    for i, letter in enumerate(self.secret_word)
)

hint_surface = self.font_msg.render(
    f"Hint: {hint_pattern}",
    True,
    (255, 210, 90)
)
screen.blit(
    hint_surface,
    (self.width // 2 - hint_surface.get_width() // 2, 230)
)

# Input box
self.input_box.render(screen)

# Submit button
pygame.draw.rect(
    screen, (50, 150, 85),
    self.submit_btn, border_radius=6
)
pygame.draw.rect(
    screen, (220, 220, 220),
    self.submit_btn, width=2, border_radius=6
)

submit_text = self.font_btn.render(
```

```
    "SUBMIT", True, (255, 255, 255)
)
screen.blit(
    submit_text,
    (
        self.submit_btn.centerx - submit_text.get_width() // 2,
        self.submit_btn.centery - submit_text.get_height() // 2
    )
)

# Hint button
pygame.draw.rect(
    screen, (130, 90, 175),
    self.hint_btn, border_radius=6
)
pygame.draw.rect(
    screen, (220, 220, 220),
    self.hint_btn, width=2, border_radius=6
)

hint_text = self.font_btn.render(
    "HINT (-0.25)", True, (255, 255, 255)
)
screen.blit(
    hint_text,
    (
        self.hint_btn.centerx - hint_text.get_width() // 2,
        self.hint_btn.centery - hint_text.get_height() // 2
    )
)
```

```
# Feedback
feedback = self.font_msg.render(
    self.feedback_msg, True, self.feedback_color
)
screen.blit(
    feedback,
    (
        self.width // 2 - feedback.get_width() // 2,
        325
    )
)
```

What this code implements

Task 1 — Fix guess validation

The guess is compared with `self.secret_word`, so typing the original word is accepted.

Task 2 — Progressive hints

Click **HINT (-0.25)** to reveal a letter in its correct position. Each hint reduces the score by 0.25 points.

Task 3 — Countdown timer

Each round lasts 20 seconds. When time expires, the answer is revealed, and the next round begins after a short delay.

Task 4 — Interactive letter tiles

> Click one tile and then another to swap their positions. Click SUBMIT to check the arranged word, or type your answer into the text box.

How to run it in VS Code

1. Open your `62_scramble` project folder.
2. Open `game/game_engine.py`.
3. Replace the existing code with the complete code above.
4. Save using `Ctrl + S`.
5. Open the VS Code terminal in the project root and run:

```
python main.py
```

If Pygame is not installed, run:

```
python -m pip install pygame
```

Important: This replacement assumes your existing `main.py` already calls `handle_event(event)`, `update()`, and `render(screen)` in its game loop, and that `TextBox` supports the same interface as in your original code.

Also test the game at your actual window size. The interface is designed around a typical window of approximately 800 × 400 pixels.

Before submitting your lab

- [ ] Test that typing the correct original word succeeds.
- [ ] Test an incorrect guess and an empty submission.
- [ ] Test multiple hints and check the score penalty.

- [ ] Wait for the timer to expire and confirm the next round starts.
- [ ] Rearrange tiles to form the answer and submit it.
- [ ] Record the required 10-second before-and-after gameplay videos.
- [ ] Submit the link to your complete ChatGPT conversation, as specified in your README.

One detail to be aware of: this code awards one point for a correct answer and deducts 0.25 points per hint. The score is never allowed to become negative. If your lab instructor expects a different scoring rule, that part can be adjusted without changing the four main features.